CSE398/498 Generative AI for Healthcare & Robotics

Lab 3a Report

Gazebo simulation: language command to semantic navigation

Xiyuan Xu | Due Oct 5, 2026 | Instructor: M. Chuah

My GitHub: https://github.com/LeoXu227/turtlebot3-gazebo-navigation-hw

What I implemented

The only program I wrote for this lab is `load()` and `infer()` in src/tb3_detector/tb3_detector/detector_core.py. Everything else in the repository is the course starter. I did not change the parser, the localizer, the map memory, or Nav2.

load() runs once at startup. It raises an error if ultralytics is missing or if the weights file is missing. Otherwise it loads `tb3det_yolo26n.pt` and moves the model onto the configured device.

infer() runs that model on one camera image. It keeps a box only when the label is in the class filter, and it returns a list of label, confidence, and box corners in the original image pixels. If `load()` was never called, it raises an error instead of pretending there were no objects.

The demo recording is report/navigation_demo.mp4 in the same repository. It shows `go to person 0` and then `go to person 2`. The figures later in this report are frames from that file, and the mp4 is also attached to this PDF.

1. How a command becomes a goal

This lab is not a language model. A short rule parser reads one text command, looks up an object the robot has already saved on its map, and asks Nav2 to drive to a pose in front of that object. The robot has to see the object first. A command sent before any matching landmark exists fails with `no active … in memory` and exploration continues.

The command is published once on `/user_command`. `parse_command()` in `tb3_query/query_core.py` lowercases the text, drops filler words (*go*, *to*, *the*, *please*), and keeps one target name plus an optional number. `semantic_targets.yaml` maps that name to the detector label. No number means the nearest saved landmark. `person 2`, `person_2`, and `person2` are the same request.

Phrases the parser accepts: `go to person`, `go to person N`, `go to trash can` (also *trashcan* or *bin*), and `go to chair`. Color and clothing are not in the grammar, so “person wearing a green shirt” and “red chair” are rejected.

The graded student code is `load()` and `infer()` in `detector_core.py`. `load()` opens the fine-tuned weights `tb3det_yolo26n.pt`. `infer()` runs that YOLO model on one camera image and returns boxes. Everything after the box (LiDAR, memory, and Nav2) is course code.

1.1 From the camera to a point on the map

Gazebo simulates the room and a TurtleBot3 Waffle Pi. SLAM Toolbox builds the occupancy map while frontier exploration drives toward unmapped edges. The camera publishes `/camera/image_raw`. `infer()` returns, for each box, a label (`person`, `trash_can`, or `chair`), a confidence, and pixel corners. The shipped threshold is 0.40. Boxes below that, and any other label, are dropped.

The localizer does not use the whole box. It takes the horizontal center, maps that pixel across the camera’s 62.2° field of view to get a bearing, then takes the median LiDAR range in a small window around that bearing. Bearing plus range becomes an (x, y) point, which is transformed into the SLAM map. A shifted box is a wrong bearing, so a bad detection sends the robot the wrong way.

One detection is not a landmark. Memory promotes a place only after repeated sightings inside about 45 seconds: **8** for a person, **12** for a chair or a trash can. Until then it is only a candidate. RViz draws a promoted

landmark as a green sphere, for example `person_0 (n=11)`. The number is how many times that spot has been seen. The id is a memory slot, not a permanent identity: the first person saved is `person_0`, the next distinct one is `person_1`, and so on.

1.2 What happens after the command

I publish the command with:

```
ros2 topic pub --once /user_command std_msgs/String "data: 'go to person'"
```

The query node picks the landmark. The coordinator leaves `EXPLORING` and enters `SEMANTIC_NAV`. The nav adapter places a goal about **0.5 m** in front of the object, facing it, so the robot stops short instead of driving into it. Nav2 drives there. On success the log says `TARGET_REACHED`, then exploration resumes a few seconds later.

`TARGET_REACHED` only means Nav2 reached its own goal. The assignment's check is stricter: the robot should finish within **1.2 m** of the true object in Gazebo. That budget covers the 0.5 m standoff, the fact that LiDAR hits the near surface rather than the center, and Nav2's own goal tolerance.

2. What I ran

I ran the person world, `warehouse_models_person`, in Docker on a Mac (Apple silicon, Rosetta, software graphics). After the robot spawned I closed the Gazebo window and used RViz. The detector loaded `tb3det_yolo26n.pt` and published person boxes around 0.95–0.99 confidence. RViz showed the green detection box and the green landmark spheres.

What worked. On one run, `go to person` selected the nearest landmark, `person_0`, and logged `TARGET_REACHED`. `go to person 2` reached `person_2` at map goal (1.62, 1.70) and also logged `TARGET_REACHED`.

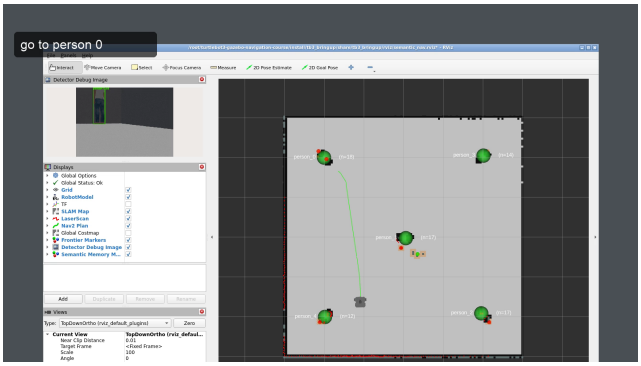
What failed. `go to person 1` was accepted three times. Each time Nav2 timed out after 60 seconds at the same approach pose (–0.30, –0.01). The robot circled the person's feet and never finished. The lab TA described this as an intermittent Nav2 problem that also happens with the reference solution, not as a bug in the detector.

Trash cans and chairs use the same pipeline, but they need 12 sightings inside about 45 seconds. On this laptop the camera only delivers a few frames per second, so those candidates expired before they became landmarks. I did not demo `go to trash can` or `go to chair`. The demo stays in the person world.

2.1 Demo: two person commands

The demo is a screen capture of the RViz desktop, about 1 minute 50 seconds. The original file is attached to this PDF as `navigation_demo.mp4` (in Acrobat or Preview, open the attachments panel, the paperclip). The figures on this page and the next are frames from that same recording, so the drives are visible even if the attachment does not play. Five person landmarks were already on the map. `go to person` selected the nearest one, `person_1`, which was only 0.06 m away, so the nav adapter skipped that goal and the robot did not move. The two commands that did drive are the ones in the video. Idle exploration between them is cut out.

`go to person 0` sent Nav2 to (–1.65, 1.65). The goal succeeded and the coordinator logged `TARGET_REACHED`. Figure 1 is the plan toward `person_0`. Figure 2 is the robot after it arrived.



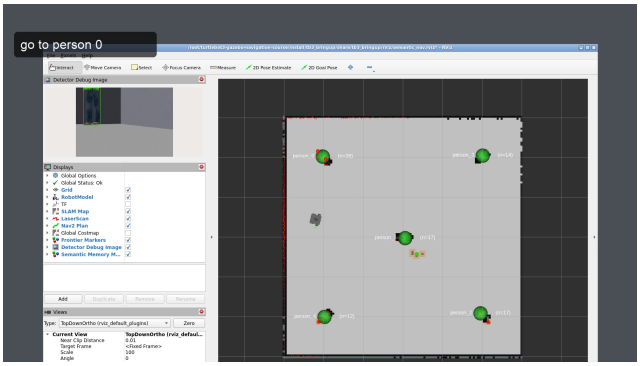


Figure 1 (left). `go to person 0`: green Nav2 plan toward person_0, with a person box in the detector view. Figure 2 (right). After `TARGET_REACHED`, the robot is next to person_0.

`go to person 2` sent Nav2 to (1.60, –1.57). That goal also succeeded and logged `TARGET_REACHED`. Figure 3 is the robot beside `person_2`. The count on that marker rose while the robot was there (n=30 in the clip), which is extra sightings of the same landmark, not a new id.

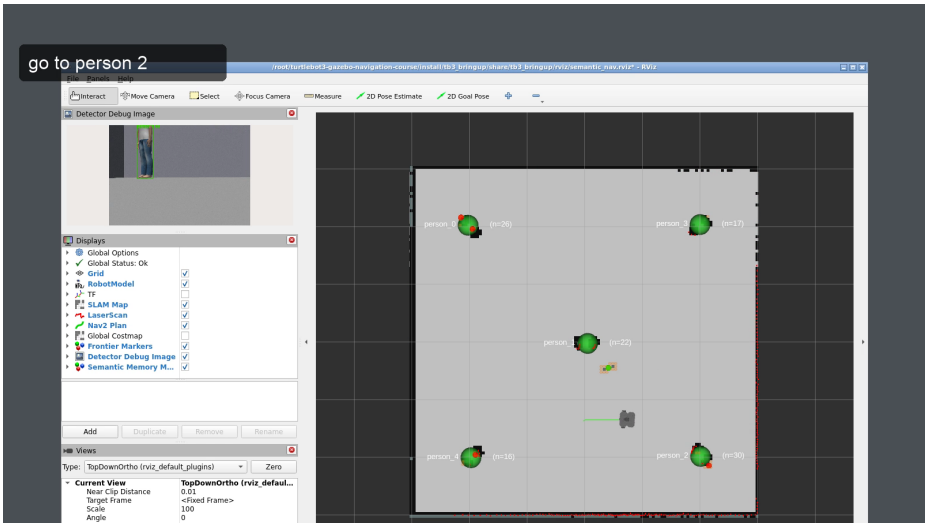
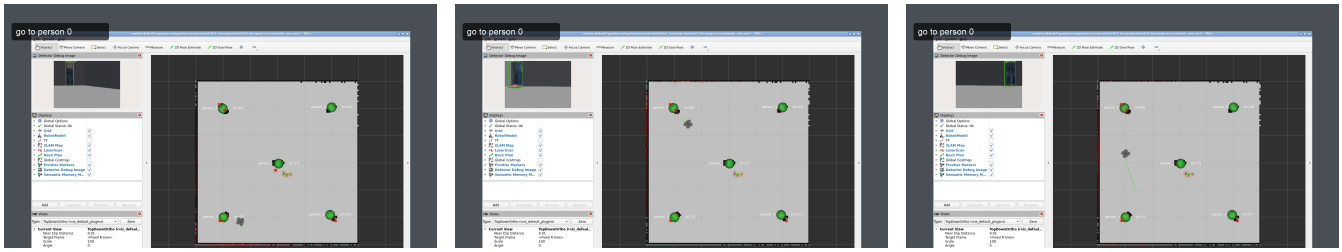


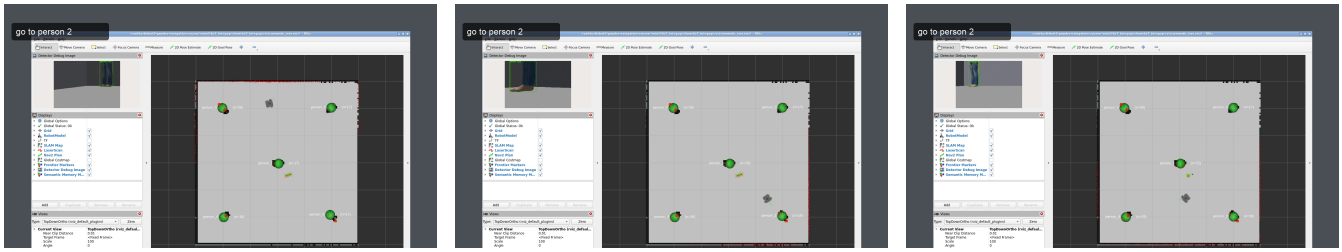
Figure 3. `go to person 2` after the goal succeeded. The robot is next to person_2. The green line is the end of the Nav2 plan.

2.2 The same demo, frame by frame

Top row: `go to person 0`, from the plan appearing to the robot arriving at person_0. Bottom row: `go to person 2`, from the new plan to the robot beside person_2.



0:04 — plan toward person_0 | 0:22 — driving to person_0 | 0:42 — arrived at person_0



0:58 — plan toward person_2 | 1:20 — driving to person_2 | 1:45 — arrived at person_2

Figure 4. Six frames from the attached demo, in time order.

3. Conclusion

A text command is a lookup on a map the robot already built. The camera and YOLO say what is in view, LiDAR turns the box into a map point, and memory keeps that point only after enough repeated sightings. Nav2 then drives to a standoff pose in front of the saved landmark. On an earlier run, `go to person` reached `person_0`. The demo video shows `go to person 0` and `go to person 2`, and both logged `TARGET_REACHED`. `go to person 1` timed out at the person's feet, which also happens with the reference solution. Chair and trash-can landmarks never formed here because the camera was too slow to collect 12 sightings in 45 seconds.

Appendix

AI tools

I used Cursor to help with the Docker/ROS setup, the detector `load()` and `infer()` implementation, report wording, and the screen capture. I ran the simulator, sent the commands, and can explain the pipeline (course AI policy).

References

1. My repository: https://github.com/LeoXu227/turtlebot3-gazebo-navigation-hw
2. Code I wrote: https://github.com/LeoXu227/turtlebot3-gazebo-navigation-hw/blob/main/src/tb3_detector/tb3_detector/detector_core.py
3. Demo video: https://github.com/LeoXu227/turtlebot3-gazebo-navigation-hw/blob/main/report/navigation_demo.mp4
4. Course starter I forked: https://github.com/YiyuchenHu/turtlebot3-gazebo-navigation-hw
5. Course packages I used as provided: tb3_query, tb3_localizer, tb3_coordinator, Nav2, SLAM Toolbox. Weights: tb3det_yolo26n.pt

How to rerun the person-world demo

- Start the course Docker stack and launch the six terminals with `world:=warehouse_models_person`. Close the Gazebo window after spawn.
- Wait until RViz shows at least `person_0` and `person_2`.
- Publish `go to person 0`, wait for `TARGET_REACHED`, then publish `go to person 2`.